

Knowledge Network — what it is, what it claims, what the code does

A document for deciding one thing: **has the product drifted from the idea, and should anything change?** It is written to be argued with. Every claim on the left is something the site or the pitch says; every note on the right is what the code actually does, checked rather than remembered. Where those disagree, the disagreement is the finding. Prepared 24 September 2026 · 53 commits · 230 tests passing.

1. The idea, in one paragraph

A person's memory is currently held by whichever product learned it. Their food app knows they are vegetarian; their travel app never will. Every AI memory product today is a store *inside* one application, so what it learns dies with it and the next product starts from nothing. This is a **knowledge network** instead: claims about a subject, each citing where it came from, written into an **ENS name the person owns**, versioned on IPFS, readable by any agent they allow. Two products share the repository — the protocol (`engine/`) and a demo built on it (`app/`). **The test of whether it works:** delete the company, and the memory survives.

2. What the product claims, and what the code does

The claim	What the code does	Verdict
Memory lives under a name you own	Wallet signs in, <code>findOwner</code> checks the ENS registry on chain, and the proven name <i>is</i> the namespace. There is no host-issued account.	True
Every claim cites its source	<code>Knowledge.sources []</code> is required on every claim; the UI shows it on every row.	True
Nothing overwrites	Commits are content-hashed with parents; <code>mergeClaim</code> combines sources and raises confidence rather than replacing.	True
Confidence means something	0.85 for a counted rule, 0.7 for a model reading the same data, 0.6 for a single observation.	True — fixed this week; it was one constant before
Agents can contribute, not commit	18 MCP tools. <code>knowledge_propose</code> opens a proposal; landing needs the reviewer role.	True
An app that never integrated can read it	Universal Resolver read, no account with us required.	True in protocol; untested by a third party
Your memory is private to you	On a hosted deployment the host generates the content key. The operator can read it.	Not yet true
You publish to ENS yourself	The host holds the publishing key. A visitor's namespace is published by us or not at all.	Not yet true

The last two are the honest gap, and they are the same gap: a hosted visitor's memory is *portable in principle* and *readable by the operator in practice*. That is precisely the critique the comparison page makes of everyone else.

3. Have we drifted?

No, with one exception that was real and has been corrected. The architecture never moved. The ENSv2 mechanisms the idea depends on are all load-bearing in code:

- **The registry as identity** — a namespace is a name someone owns, confirmed by the chain rather than by our database.
- **Subregistries and `setParent`** — `food.yours.eth` has its own owner and policy; a branch can be given away without giving away the rest.
- **EnhancedAccessControl** — 55 role constants across 32 nybble-packed slots, granted and revoked by transaction. A reviewer's authority is verifiable, not asserted by our server.
- **`contenthash` on the resolver** — versions are pointed at, not stored by us.
- **Universal Resolver** — anyone can read without asking us.

What did drift was the story, not the system. For a period the landing page stopped saying any of the above, which made the product read as "a memory app with a wallet button". It also illustrated the use case with Wikipedia, and later with Food / Shopping / Travel icons — categories that describe a wish rather than a product, because DoorDash, Amazon and Airbnb publish nothing a person can use to read their own history. Both are fixed. Six ENSv2 mechanisms are now stated on the landing page, and every source named is one that genuinely connects.

4. What actually connects

Researched against vendor documentation, with four MCP endpoints checked live for OAuth metadata.

Working today

Source	What it reads	How
GitHub	Repositories you contribute to, and what you write them in	OAuth
Granola	Who you meet with	MCP · OAuth + PKCE + dynamic registration
Google Workspace	Recurring meetings, services that mail you	OAuth
Google Takeout	Where you order from, channels you follow	Data Portability API
Linear	Teams and projects you are assigned work in	OAuth
Browser, editor, shell, Claude Code	Local files, on your own machine	no network

Reachable, not yet wired

Strava (OAuth **and** an official read-only MCP connector — the closest existing proof that this pattern works), Spotify, Notion, Slack, Todoist, Oura, Whoop, Last.fm.

Closed, and why it matters

There is **no consumer OAuth for order history** at DoorDash, Uber Eats, Deliveroo, Zomato, OpenTable or Amazon — every API in those spaces is merchant-side. Instagram switched off its personal API in December 2024. Triplt closed to new integrations in February 2026. LinkedIn's self-serve scopes return a name, not a history. Regulation does not open these doors. GDPR Article 20's "where technically feasible" has been a dead letter for a decade, and the EU Data Act's real-time clause reaches connected devices rather than order histories. **Google's Data Portability API is the one exception**, and it exists because the DMA compelled it. One OAuth relationship covers food orders and reservations, shopping, Maps, Play purchases, and YouTube watch history — which the YouTube Data API has refused to return since 2016. It is how food and shopping actually arrive, and it is now implemented.

5. Who the competition is for

This is the sharpest distinction and it is not the one usually drawn.

	Who the customer is	Where the person signs in
Mem0	The developer. No app connectors at all — its 22 "integrations" are frameworks (LangChain, CrewAI), not apps.	Nowhere. The developer supplies the data.
Supermemory	The developer, but with 7 OAuth connectors the developer embeds and their end user authorises: Drive, Gmail, Notion, OneDrive, GitHub, Granola, web crawler.	Into the developer's product.
Instinct	The person. Invite-only; no published connector list . Reaches services through stored credentials and browser automation on a persistent cloud machine.	Into the assistant.
This	The person.	Nowhere. They prove a name they already own, and the namespace is theirs before any app connects.

They are not lying about persistence. Memory that survives a restart is real and it works. The question they cannot answer is *persistent to whom* — and for Mem0 and Supermemory the answer is: to the application, because the application is the customer and the person is the subject of the record rather than a party to it.

6. Two ways a claim gets in

Both end in a proposal the owner reviews. This was the idea from the start and the page had stopped saying it. **1 — You connect an app.** Grant read access to something you already use. It observes patterns, never contents, and proposes what it learns. The consent screen is *generated from the thresholds the reader actually uses*, so it cannot promise one thing and do another. **2 — An agent writes to you.** Any assistant speaking MCP can propose to a namespace you own. It does not integrate with us and we never see its data — it resolves the name, proposes, and the namespace policy decides. This is what makes it a network rather than another connector product. Both paths run automated findings first — duplicates, contradictions, claims with no source — and then a person decides. **Contributing and committing are deliberately different things.**

7. Where the boundary sits

Layer	Learns	Acts
engine/ — the protocol	yes	never . All 18 MCP tools are about knowledge; nothing books, sends or replies.
The watcher	yes	never . It reads sources and writes claims, and that is all it can do.
Chat	yes	with approval . Writes to connected apps stop and show the exact tool call.

An agent that watches everything is tolerable *because it cannot act on what it sees. *One that acts is tolerable* because you asked it to in the moment*. Collapsing those gives you something that does both silently. One safety detail worth knowing: the read/write classifier defaults to **"this writes"** and checks write verbs before read prefixes. `list_and_close_stale` starts with a read prefix and closes issues; classifying it as safe would have had the agent acting unasked.

8. What is left

Before this can claim the word "decentralized":

- **Content keys derived from a wallet signature**. Today the host generates them, so a hosted visitor's memory is readable by the operator. One day's work, and it is the difference between a pitch and a property.
- **The browser signs** `setContenthash`. Publishing becomes the user's transaction rather than ours. Costs testnet gas, which is free from faucets, and needs labelling — the current *"signing costs nothing"* copy is accurate only because this step does not exist yet.

Before real users:

- **Persistence**. Repositories, grants, activity and tokens are JSON files in `~/.recall`. Needs a database behind the existing `RepoStore` interface.
- **Publish the engine**. Five packages build, have READMEs, and `@recall` is free on npm. `npm publish` is the remaining step.

Smaller: `T00LS` in `chrome.ts` is still 8 host patterns; `TOPIC_RULES` still classifies by keyword, though it now falls back to `notes` rather than asserting a decision; `repoview.ts` mixes loading with fifteen selectors.

9. Questions worth arguing about

These are genuine, not rhetorical. The answers would change what gets built next.

- **Is "the person is the customer" a market or a principle?** Every product listed above chose the developer, and they are not stupid. If the answer is principle, the roadmap is right. If it is market, the engine should be sold to developers and the demo is a reference implementation.

- **Does the review step survive contact with volume?** It is the argument — contributing and committing are different — and it is also friction. Personal namespaces default to `approvals: 0` precisely because gating your own claim is theatre. At what volume does that default become wrong?
- **Is Sepolia enough for a real user?** Names are free and the demo works. A person who wants their memory to outlive us needs mainnet, and that is a cost somebody pays.
- **Should the page keep saying what is closed?** It currently names DoorDash, Amazon and Instagram as doors that will not open. It turns the gap into the argument, and it also tells a judge exactly where to push.
- **Is the demo a product or a proof?** It is currently good enough to use, which raises the question of whether it should be supported. If it is a proof, some of the roadmap is wasted effort.

Appendix — verification

Everything in this document was checked rather than recalled.

```

engine/core      75 tests    objects, merge, review, roles, crypto
engine/storage   14 tests    IPFS adapters, contenthash encoding
engine/repo      23 tests    commits, branches, proposals, review
engine/cli       7 tests     memory-export importer
engine/mcp       5 tests     the 18-tool server, spawned for real
app/connect      106 tests   sources, OAuth, tokens, chat, policy
-----
                230 passing

```

Four structural checks run in CI and fail the build:

- `check:layering` — `engine/` may not import from `app/`. A protocol that needs its own demo is not a protocol.
- `check:packaging` — published entry points exist, and every resolver prefers source.
- `check:config` — no placeholder identity (`acme.eth` , `demo.eth`) in code that runs for a real person.
- `check:abi` — every contract call exists in a vendored ABI.

Live on Ethereum Sepolia: `recalltest.eth` , `worldhistory.eth` , `india.worldhistory.eth` , `rishtokyo.eth` — the last registered through the app's own browser flow, wallet-signed, as a test of the onboarding path.